AI Agents as Knowledge Explorers: A Case Study of 398 U.S. Case Law Quotations

Jeremiah Milbauer
Carnegie Mellon University
and Mozilla AI
jmilbaue@andrew.cmu.edu

Abstract

Are AI research agents capable of exploring knowledge corpora to find information? We introduce ExTraBench, an evaluation of AI agents' exploration trajectories in a citation-linked corpus of U.S. Supreme Court cases. Despite the corpus' prominence and thus high likelihood of prior observation through pretraining data, LLM agents struggle to find information in even moderately challenging exploration tasks of 2+ hops.

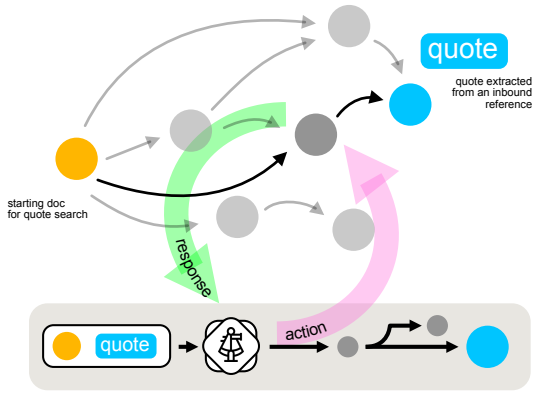

Figure 1: An illustration of the ExTraBench navigation task: given a quote and a starting document in the citation graph, a research agent must find a path to the target document containing the quotation by interacting with the environment.

1 Introduction

Open-ended exploration of text corpora is fundamental for producing new knowledge, finding new ideas, and validating old ones. However, when artificial intelligence (AI) agents are deployed as information access tools, as in OpenAI's Deep Research [1], Google's Gemini DeepResearch[2], and AI2's OpenScholar (Asai et al., 2024), they are rarely designed or evaluated based on a capability for *exploration*: the ability to accurately navigate an open-ended environment without relying on an offline index of the environment itself.

These systems use large language models (LLMs) as tool orchestrators (Yao et al., 2023; Asai et al., 2024) capable of iteratively retrieving or employing search engines, reading documents, and synthesizing results, and research-oriented evaluations are typically focused on scientific applications such as literature review and synthesis or data analysis; see AstaBench (Bragg et al., 2025). Yet research is usually more open-ended, the interactions and affordances with the corpus more complex, and the areas of interest inclusive of domains well beyond scientific literature (Gururaja et al.; Athukorala et al., 2013; Abbott, 2014).

In this work, we emphasize exploration as a core research capability, and broaden the scope of research agent evaluation to include domains of scholarship beyond scientific literature.

Concretely, in this work we present ExTraBench (**Ex**ploration **Tra**jectory **Bench**mark), the first direct evaluation benchmark for research exploration in an open environment, derived from the Caselaw Access Project (CAP) corpus of legal opinions.[3]

First, we construct approximately 400[4] quotation search tasks. Given a corpus of legal opinions D, and a directed citation graph G, each task consists of an arbitrary starting document $u \in D$ and a significant excerpt q of a target document $d \in D$, such there exists a path P_i from u to the document d containing q. The agent must navigate $\langle D, G \rangle$ to find d. In these tasks q (which guides the search) is an excerpt of d; performance on this research proxy can be considered an upper bound on agentic

[1] https://openai.com/index/introducing-deep-research/
[2] https://blog.google/products/gemini/google-gemini-deep-research/
[3] https://case.law/
[4] More questions than other literature understanding datasets, see: (Bragg et al., 2025)

navigation with less concrete guidance (i.e., where it is harder for an agent to verify and terminate).

Building on top of Mozilla.ai's `any-agent` library, we evaluate leading LLMs within different agentic frameworks, and assess their ability to efficiently navigate the corpus and find the quotation. We find that even for relatively simple search tasks in this well-attested domain, state of the art LLMs struggle to find the quotations. We intend to expand the dataset, and publish a leaderboard and benchmark.

2 Defining Open-Ended Exploration

We define a *research environment* as a collection of documents containing informative text and metadata. This metadata could include features like author lists, citation counts, peer reviews, and citation links to other documents. A research environment can be *closed*, in which case all documents are known to an agent ahead of time, or *open*, in which case the full set of documents is not known to the agent but can still be discovered.

In this work, we focus on the task of *agentic exploration*, measuring the ability of an LLM agent to accurately and efficiently navigate an *open* knowledge corpus without being provided either (1) information about the overall structure; or (2) the ability to search the entire corpus. Instead, the agent must navigate links between documents to find information.

We define a *research agent* as a system capable of receiving a query from the user, sequentially producing actions (be they text output or tool calls), and ultimately returning a response. We restrict the available tools to navigational tools: for a given document, either listing outgoing citations or reading the text. To understand the ability of a given agent to navigate the corpus, we assess: (1) whether the agent finds the information corresponding to the task; and (2) the efficiency of its actions with respect to the difficulty of the task within the citation graph. Evaluation is discussed in Section 4.3.

3 Benchmark

3.1 Tasks

The dataset consists of 398 quotation search tasks, denoted by $\mathcal{T}$, defined over a citation graph $\langle D, G\rangle$ of U.S. Supreme Court cases. Here D is the set of documents (court cases), and $G \subseteq D \times D$ is the directed citation graph, where $(d, d') \in G$ indicates that document d cites document d'.

Each task $T_i \in \mathcal{T}$ is a pair $T_i = (q_i, P_i)$, where q_i is a quotation extracted from a target document $d_i \in D$, and $P_i = \langle d_{i,1}, d_{i,2}, \ldots, d_{i,n} = d_i\rangle$ is a path in G consisting of n documents, terminating at d_i. The path P_i satisfies the condition that for every document $d_{i,j} \in P_i$, P_i is the shortest path in G from $d_{i,j}$ to the quotation-containing document d_i.

3.2 Construction

To construct the evaluation tasks and the research environment, we use the Caselaw Access Project, a publicly available dataset that contains the full text and citations for millions of US court opinions. Court opinions are written decisions produced by courts that establish *precedent*: legal interpretations and principles that must be followed by courts in the future. When issuing an opinion, the courts will often cite arguments or precedent established by previous cases with similar jurisdiction. We narrow the scope of our evaluation to just the US Supreme Court (SCOTUS),

Dataset construction has two phases: quotation selection, which leverages the citation graph to identify salient excerpts of documents in the corpus, and path generation, which automatically generates search paths of varying difficulty levels.

Quotation Selection Given a document in the corpus $d_i \in D$, and a list of documents which cite d_i, $\text{Inbound}(d_i) = \{d_j \in D \mid (d_j, d_i) \in G\}$, we identify the exact text of referring passages $\text{RP}(d_i)$ which contain the references to d_i. For each $r \in \text{RP}(d_i)$, we compute a sequence alignment between the referring text r and the full contents of d_i. We use the Smith-Waterman algorithm (Smith et al., 1981), denoted $\text{SW}(\cdot)$. For a given document d_i, we produce a set of candidate excerpts:

$$\text{Excerpts}(d_i) = \{\text{SW}(r, d_i) | r \in \text{RP}(d_i)\}$$

To identify high-quality excerpts across all cited cases, we use the *Jaccard Index* between the excerpt text and its citation context: $J(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$, where X and Y are sets of words. We retain the top 2% of excerpts ranked by $J(e, d)\ \forall e \in$ Excerpts(d), and from these we keep the most concise half (shortest length). This yields a pool of 460 excerpts for the CAP SCOTUS data. We denote this set of quotations Q, where q_i is a quotation from d_i.

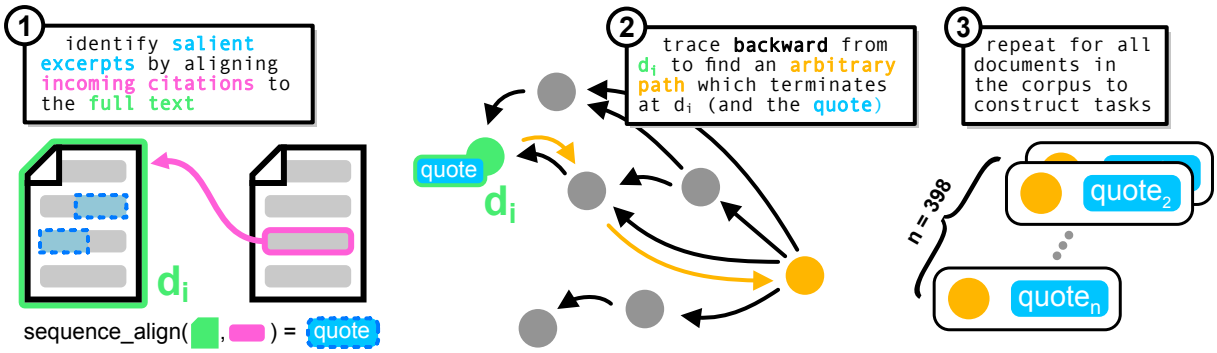

Figure 2: The stages of task construction in ExTraBench. (1) Given a document d_i, identify its salient excerpts by aligning the full document with text from other documents' passages that cite d_i. (2) Starting from d_i, traverse backward through the citation graph to construct arbitrary paths of specific lengths which (forward) terminate at d_i.

Path Generation For each excerpt quote q_i from d_i, we trace a path backward through the citation graph for n steps ($|P_i^{-1}| = n + 1$), to form a variable-difficulty quotation search task. Difficulty is variable because the forward path P_i can be prefix-truncated to begin at a document of any distance $\leq n$ from d_i; thus $P_i[n-1:]$ is a one-hop path $\langle d_{i,n-1}, d_{i,n} = d_i \rangle$, and so on.

To find these paths, we sample random walks backward through the graph for each excerpt node d_i, until we find a path of length $n = 4$ that is also a shortest path, or attempt 1000 times. We do not find such a path for all excerpt nodes; this process yields 398 search tasks.

4 Preliminary Experiments

Using `ExTraBench`, we evaluate both the efficacy and the efficiency of a research agent in exploring the research environment to locate a given quotation. We use a random subset of the generated search tasks.

4.1 Research Agent Configurations

Each research agent is specified by the underlying LLM and the agent framework used. We considered three versions of recent OpenAI LLM APIs: `gpt-4o`, `gpt-4.1`, and `gpt-5`, and four different agent frameworks (detailed below), for a total of 12 research agent configurations.

LangGraph Part of LangChain[5], a popular library for LLM application development. Implements a ReAct-style agent: (1) The LLM is provided with history and queries; (2) The LLM generates text or tool calls; (3) if the LLM generates tool calls, the results of the tool are added to the history; (4) repeat (Yao et al., 2023).

LlamaIndex LlamaIndex[6] is another framework which implements a ReAct-style agent.

OpenAI OpenAI Agents SDK [7] includes capabilities for tools, agent swapping, and guardrails.

TinyAgent TinyAgent[8] is the baseline agent implementation from Mozilla AI, inspired by the Huggingface Tiny Agents framework [9]

We use `any-agent`[10], an AI agent library from Mozilla.ai to run comparable tests with each of these frameworks and LLMs. `any-agent` smooths the differences in these frameworks and allows us to analyze the output agent traces.

4.2 Tasks Used for Preliminary Evaluation

We evaluate each agent configuration on 100 traversal tasks of path length 4, as well as 200 additional corresponding tasks of easier difficulties (path lengths 3 and 2), using the `-mini` version of each model. Evaluation for full size models is very expensive, so we use a subset of 30 quote search tasks to evaluate the full-sized LLMs. We set a 10 minute time limit.

4.3 Preliminary Evaluation Results

For each task, we evaluate in terms of precision, recall, and success of the research agent.[11] Precision

[5] https://www.langchain.com

[6] https://www.llamaindex.ai/

[7] https://github.com/openai/openai-agents-python

[8] https://mozilla-ai.github.io/any-agent/agents/frameworks/tinyagent/

[9] https://huggingface.co/blog/tiny-agents

[10] https://mozilla-ai.github.io/any-agent/

[11] After evaluation, we find that the model has a statistically significant impact on the outcome, but the framework only

Model	Dist.	Prec.	Rec.	Hit (%)	Timeout (%)	Match (%)
gpt-4.1	1	0.64	1.00	0.94	0.06	0.53
	2	0.17	0.51	0.29	0.32	0.30
	3	0.14	0.33	0.12	0.46	0.22
gpt-4o	1	0.45	0.97	0.92	0.06	0.49
	2	0.09	0.44	0.15	0.22	0.23
	3	0.11	0.34	0.04	0.32	0.16
gpt-5	1	0.81	0.91	0.68	0.25	0.43
	2	0.34	0.43	0.29	0.33	0.36
	3	0.14	0.17	0.00	0.33	0.35

(a) Full-size models

Model	Dist.	Prec.	Rec.	Hit (%)	Timeout (%)	Match (%)
gpt-4.1-mini	1	0.54	0.84	0.79	0.06	0.40
	2	0.14	0.22	0.08	0.16	0.14
	3	0.10	0.14	0.01	0.24	0.07
gpt-4o-mini	1	0.34	0.95	0.86	0.11	0.39
	2	0.09	0.37	0.08	0.38	0.08
	3	0.09	0.27	0.01	0.38	0.04
gpt-5-mini	1	0.71	0.93	0.67	0.28	0.34
	2	0.22	0.37	0.11	0.49	0.14
	3	0.10	0.13	0.01	0.55	0.08

(b) -mini models

Table 1: Precision, recall, hit rate, timeout rate, and text match percent in the final answer for search trajectories, sorted by model and the number of hops required in the search task. Left table shows full-size models, right table shows `-mini` size models. Results are aggregated across four agent frameworks run via `any-agent`.

is calculated as the percentage of visited documents that are part of the true path. Recall is calculated as the percentage of the true path visited by the agent. We consider two kinds of success: whether the model *visited* the correct document at some point, and whether the model output the name of the correct case in the final response (measured as % of the correct document title in the final response, excluding stopwords).

Accuracy We find models often successful for single-hop traversal tasks, but that precision, recall, and success rate drops dramatically for tasks that require traversal of distances > 1. Table 1 shows results for the `-mini` and full sized models. **Precision** is the percentage of visited documents that fall within the minimal path to the target document. **Recall** is the percentage of documents in the minimal path which were visited by the agent. **Hit Rate** is the percentage of runs which return the correct document. In some cases – especially with the larger models - the search process fails to finish within a 10 minute timeout window. These instances are treated as failures. We also see that some large models can achieve text match in the final response without actually visiting the document, likely owing to their parametric memory and exposure to the quote during training. However, responses of this type are not grounded in actual documents.

Trajectories Figure 3 shows the distribution of the number of documents visited by each model (across frameworks) and each framework (across models). All configurations were given the same time limit. We see that some (e.g., `gpt-4o-mini`), visit many documents as they search through the corpus. An important capability for a research

sometimes does. Given space constraints, we averaged results across frameworks in results tables.

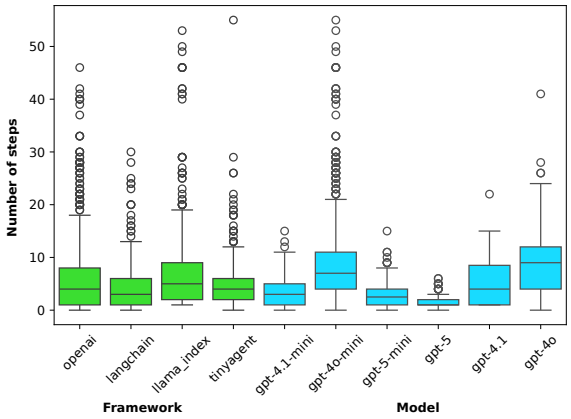

Figure 3: Distribution of the number of steps taken to complete the task, by Framework and by Model. Boxes represent the 25-75 range. Tasks which timed out are not shown.

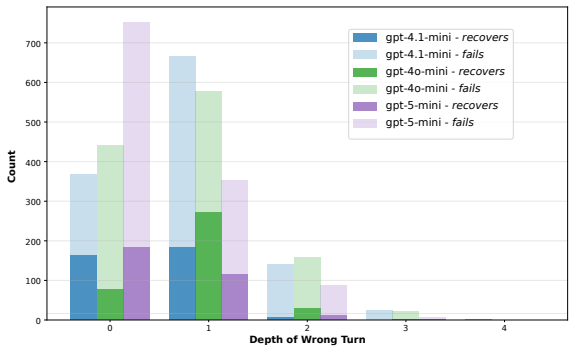

Figure 4: Recovery rates by error depth for `-mini` models. Some runs have maximum error depth of 0, either because they make no unrecoverable errors, or because the trajectory itself is too short.

agent is the ability to recover from an incorrect trajectory. For each run, we measure the *depth* (distance from the starting point) of the worst *wrong turn* (reading a document which has no viable path to the target document), and whether the research agent was able to recover (find the target document in the end). Figure 4 shows recovery rates for different models at different depths of worst wrong turn.

5 Conclusion and Next Steps

Initial results demonstrate that `ExTraBench` constitutes a challenging yet tractable setting for evaluating the capabilities of AI agents. Our long-term objective is to develop `ExTraBench` into a live, publicly accessible benchmark to support evaluation of research agent navigation capabilities.

Expanding the Evaluation A key direction for future work is the identification of additional datasets, and particularly scholarly corpora beyond legal case law, that can support task designs aligned with the same framework of navigation-based information exploration. We aim to investigate alternative task formulations as well, such as those which which present the agent with less concrete objectives or where the trajectories themselves are based on more natural browsing patterns. We are also interested in identifying other types of research browsing behavior that can be formulated as agent evaluation tasks.

Human-Centric Perspective `ExTraBench` motivates an interest in understanding the navigation patterns of humans, of research agents, and of human-agent collaborative research systems. Ultimately, we would like to conduct user studies to better understand how human researchers approach the tasks represented in `ExTraBench`, including how individuals with varying levels of expertise or varying access to research tools navigation a corpus when searching for speciic information. These studies could inform the design of future sensemaking tools that incorporate research agents.

Acknowledgments

Sincere thanks to John Dickerson, Emma Strubell, Fernando Diaz, Nathan Brake, Davide Eynard, and the Mozilla AI team for helpful discussions. This work was made possible by the *Builder in Residence* program at Mozilla AI.

References

Andrew Abbott. 2014. *The system of professions: An essay on the division of expert labor*. University of Chicago press.

Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, ..., and Hannaneh Hajishirzi. 2024. Openscholar: Synthesizing scientific literature with retrieval-augmented lms. *Preprint*, arXiv:2411.14199.

Kumaripaba Athukorala, Eve Hoggan, Anu Lehtiö, Tuukka Ruotsalo, and Giulio Jacucci. 2013. Information-seeking behaviors of computer scientists: Challenges for electronic literature search tools. *Proceedings of the American Society for Information Science and Technology*, 50(1):1–11.

Jonathan Bragg, Mike D'Arcy, and Asta Team at Allen Institute for AI. 2025. Astabench. https://github.com/allenai/asta-bench.

Sireesh Gururaja, Nupoor Gandhi, Jeremiah Milbauer, and Emma Strubell. Beyond text: Characterizing domain expert needs in document research. In *Findings of the Association for Computational Linguistics: ACL 2025*.

Temple F Smith, Michael S Waterman, et al. 1981. Identification of common molecular subsequences. *Journal of molecular biology*, 147(1):195–197.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

A Copyright notice

Preprint. © 2025 Jeremiah Milbauer. This preprint is made available under the Creative Commons Attribution License (CC BY).